

		INSTITUTO FEDERAL DO NORTE DE MINAS GERAIS <i>Campus Montes Claros</i> Bacharelado em Ciência da Computação		
Disciplina: PDI	Atividade: Exercício	Professor: Wagner Ferreira de Barros	Valor: 5pts	Nota:
Data: 13/08/2024	Aluno(a):			

INSTRUÇÕES

- Esta Lista de Exercícios é INDIVIDUAL.
- Todo e qualquer material didático online ou pessoal poderá ser utilizado para a realização desta atividade (e é fortemente aconselhado que seja utilizado). Entretanto, todo material bibliográfico consultado deverá ser citado.
- **Preferencialmente, desenvolva todo seu trabalho em um único arquivo no formato Jupyter Notebook.**
- O exercício deverá ser iniciado na aula da terça-feira e finalizado até o dia posterior, quarta-feira (preferencialmente no horário de aula).
- **Cópias de trabalhos feitos por seus colegas não serão admitidas e, se identificadas pelo professor, serão punidas com anulação TOTAL da nota de ambos os trabalhos.**

1 Filtros de Suavização Linear

1.1 Comparação entre Filtros de Suavização

Escolha uma imagem com diferentes tipos de ruído (por exemplo, sal e pimenta e ruído gaussiano). Aplique os seguintes filtros de suavização: média, mediana e média ponderada. Compare os resultados e discuta qual filtro é mais eficaz para cada tipo de ruído.

Código 1: Exemplo de Leitura e Aplicação de Filtros

```

1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Carregar a imagem com ruído
6 imagem = cv2.imread('imagem_ruidosa.jpg', cv2.IMREAD_GRAYSCALE)
7
8 # Filtros a serem aplicados
9 # (Adicionar código para aplicação dos filtros e visualização dos resultados)

```

1.2 Filtro da Média Ponderada

Implemente um filtro da média ponderada com diferentes configurações de pesos. Aplique este filtro em uma imagem e compare os resultados com o filtro de média simples. Discuta como a variação dos pesos afeta a suavização da imagem.

Código 2: Exemplo de Filtro de Média Ponderada

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Carregar a imagem
6 imagem = cv2.imread('imagem.jpg', cv2.IMREAD_GRAYSCALE)
7
8 # Definir e aplicar o filtro de média ponderada
9 # (Adicionar código para aplicação do filtro e visualização dos resultados)
```

2 Filtros de Detecção de Bordas

2.1 Comparação entre Filtros de Detecção de Bordas

Aplique os seguintes filtros de detecção de bordas em uma imagem: *Roberts*, *Prewitt*, *Sobel* e *Laplaciano*. Compare os resultados em termos de nitidez das bordas detectadas e discuta qual filtro seria mais adequado para diferentes tipos de imagens e cenários.

Código 3: Exemplo de Aplicação de Filtros de Bordas

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Carregar a imagem
6 imagem = cv2.imread('imagem.jpg', cv2.IMREAD_GRAYSCALE)
7
8 # Aplicar os filtros de detecção de bordas
9 # (Adicionar código para aplicação dos filtros e visualização dos resultados)
```

2.2 Escolha do Filtro de Bordas

Considere uma imagem com bordas finas e uma com bordas grossas. Aplique os filtros de *Roberts*, *Prewitt*, *Sobel* e *Laplaciano* em ambas as imagens. Discuta qual filtro oferece os melhores resultados para cada tipo de borda e por quê.

Código 4: Exemplo de Aplicação de Filtros em Imagens Diferentes

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Carregar as imagens
6 imagem_fina = cv2.imread('imagem_bordas_finas.jpg', cv2.IMREAD_GRAYSCALE)
7 imagem_grossa = cv2.imread('imagem_bordas_grossas.jpg', cv2.IMREAD_GRAYSCALE)
8
9 # Aplicar os filtros de detecção de bordas
10 # (Adicionar código para aplicação dos filtros e visualização dos resultados)
```

2.3 Efeito da Intensidade do Filtro

Aplique o filtro *Laplaciano* com diferentes intensidades (ajustando o *kernel*). Compare os resultados em termos de destaque das bordas e discuta como a intensidade do filtro pode influenciar a detecção de bordas em imagens com diferentes níveis de detalhe.

Código 5: Exemplo de Aplicação do Filtro Laplaciano com Intensidades Diferentes

```
1 import cv2
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Carregar a imagem
6 imagem = cv2.imread('imagem.jpg', cv2.IMREAD_GRAYSCALE)
7
8 # Definir diferentes kernels do Laplaciano
9 # (Adicionar código para aplicação dos kernels e visualização dos resultados)
```